

Introduction to Computer Simulation

Home Work 3

Guilherme Schewtschik (3748052)

November 10, 2025

Question 1

1.a)

For question number one the following code was used along side the code from the previous homework:

```
def get_Energy_all_States(N,J, h=0):
    #set necessary arrays
    state = get_States(N)
    Energy_interaction = np.zeros(len(state))
    Magnetization = np.sum(h*get_States(N), axis = 1)

    for j in range(0,len(state)):
        Energy_j = 0

        #Assigns the energy of state j into the jth element of the Energy array
        for i in range(0,len(state[0])-1):
            Energy_j += state[j][i]*state[j][i+1]

        Energy_interaction[j] = Energy_j

    Energy_total = -J*Energy_interaction - Magnetization

    return Energy_total, -J*Energy_interaction , Magnetization

def get_Omega(A):
    #returnts unique values in A and their frequency
    return np.unique(A, return_counts=True)

def get_Mean(A, beta):
    A_unique, Omega = get_Omega(A)

    return np.sum(A_unique*Omega*(np.exp(-np.outer(beta, A_unique))), axis =1)
    /np.sum(Omega*(np.exp(-np.outer(beta, A_unique))), axis =1)
```

Using the function:

```
E_t, _, _ = get_Energy_all_States(20,1,0)
```

```
beta = np.arange(0.5,40,0.01)
E_t_mean = get_Mean(E_t,beta)
```

```
C = np.gradient(E_t_mean, 1/beta)
```

Plotting this in the interval $T \in (0, 2]$

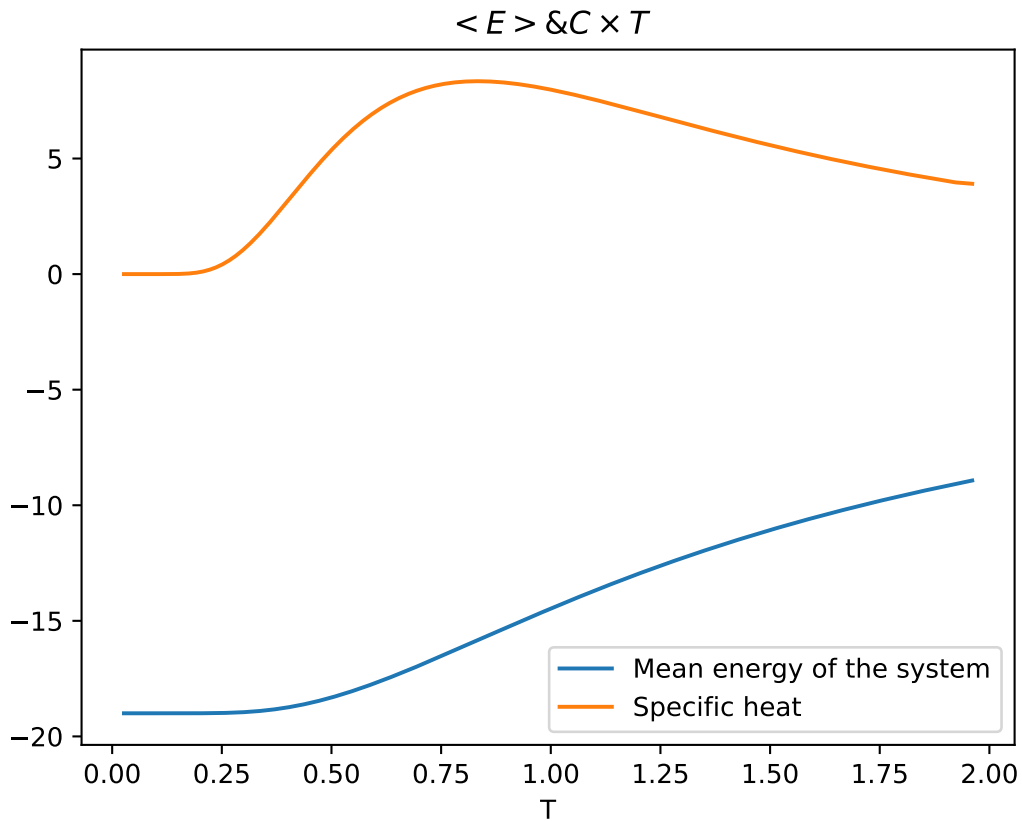


Figure 1: Plot of $\langle E \rangle$ & C as a function of T for $k_B = 1$ & $h = 0$

1.b)

Similarly:

```
E_t_M, _, M = get_Energy_all_States(20,1,0.5)
```

```
beta = np.arange(0.5,40,0.01)
```

```
E_t_M, M = [get_Mean(E_t,beta), get_Mean(M,beta)]
```

Gives us the following plot:

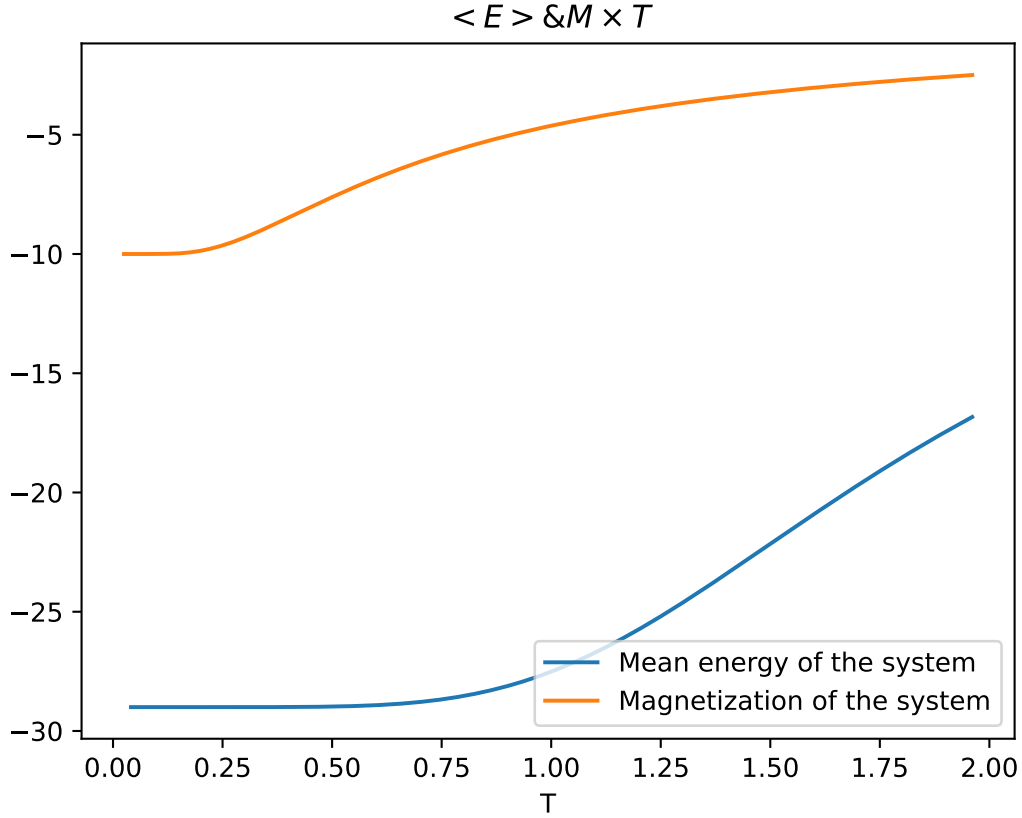


Figure 2: Plot of $\langle E \rangle$ & $\langle M \rangle$ as a function of T for $k_B = 1$ & $h = 0.5$

1.c)

In a system with N spins $s_i = \pm 1$ with no external field. The Hamiltonian is

$$H = -J \sum_{i=1}^{N-1} s_i s_{i+1}, \quad (1)$$

We can define a number K such that:

$$k = \#\{i : s_i \neq s_{i+1}\}, \quad k = 0, 1, \dots, N-1. \quad (2)$$

The energy of a spin configuration can be expressed in terms of k :

$$E = -J(N-1-k) + Jk = -J(N-1) + 2Jk. \quad (3)$$

We can see that the minimum energy occurs for $k = 0$ (all spins aligned):

$$E_{\min} = -J(N-1), \quad (4)$$

So we have $\binom{N-1}{k}$ ways to do this. As the orientation of spins is only important with respect to its neighbors we can just flip the sign to get another set of configurations, giving a factor of 2 to the expression. Therefore, we conclude that:

$$\Omega(E) = 2 \binom{N-1}{k}, \quad k = \frac{E - E_{\min}}{2J}. \quad (5)$$

If E does not satisfy $E = E_{\min} + 2Jk$ with integer $k \in [0, N - 1]$, then $\Omega(E) = 0$.

Using the python script to check $\Omega(E)$ for $N = 4$:

```
E_t , E_in, M = get_Energy_all_States(4,1,0)
E_t_unique, degeneracy = get_Omega(E_t)
```

We get the arrays:

```
E_t_unique = [-3., -1., 1., 3.]
degeneracy = [2, 6, 6, 2]
```

So we can calculate:

```
degeneracy_analitic = 2*sp.binom(3,(E_t_u-np.min(E_t_u))/2)
```

Which gives us:

```
degeneracy_analitic = [2. 6. 6. 2.]
```

Confirming that the program we wrote is correct.

Question 2

2.a)

We can estimate the area of the circle by analyzing the probability of a point falling inside the radius:

$$P = N_{inside}/N \quad (6)$$

We can code this in a few lines:

```
r = np.random.rand(2,N)*2-1
r_mag = np.sqrt(r[0]**2 + r[1]**2)

mask = r_mag<=1

p = len(r[0,mask])/N
```

Plotting the points we get:

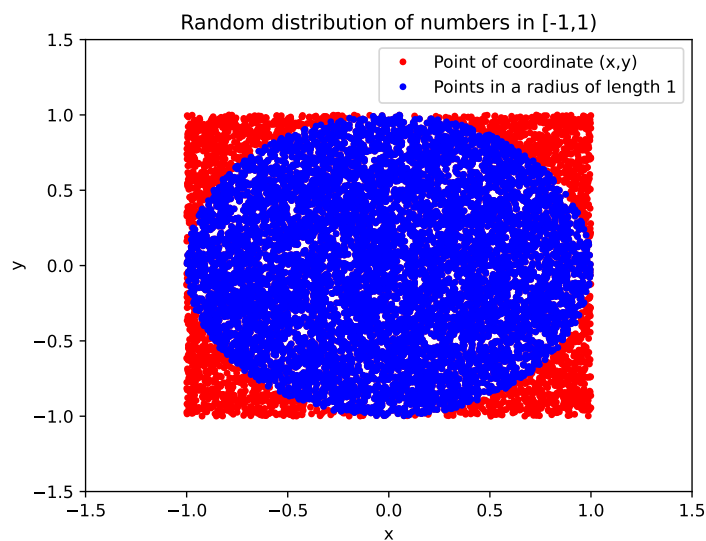


Figure 3: Plot of 10^4 random points

We can retrieve the area of the circle by multiplying the probability of a point being inside the circle by the area of the square.

With this we arrive at an Area of 3.1404, which is close to the expected area of π .

Given the random nature of this process the result is going to vary every time we run the process, for a more consistent result we can average the result of multiple runs:

```
iterations = 1000

error = np.zeros(iterations)
area = np.zeros(iterations)

for i in range(0,iterations):
    r = np.random.rand(2,N)*2-1
    r_mag = np.sqrt(r[0]**2 + r[1]**2)

    mask = r_mag<=1

    p = len(r[0,mask])/N

    error[i] = (p*A_s-A_c)/A_c
    area[i] = p*A_s

error_mean = np.mean(error)
area_mean = np.mean(area)
```

So the final result is:

$$A = 3.1415776 \quad \& \quad Error = -4.79170 \times 10^{-6}$$

2.b)

The efficiency of the sampling is equal to:

$$\eta = \frac{A_{ellipse}}{A_{total}} = \frac{\pi ab}{4a^2} = \frac{\pi b}{4a} \quad (7)$$

For $a = 1$ then $\eta = \pi b/4$, in order for the efficiency to be bigger then 0.5 then:

$$b \geq \frac{2}{\pi} \quad (8)$$

Plotting the points:

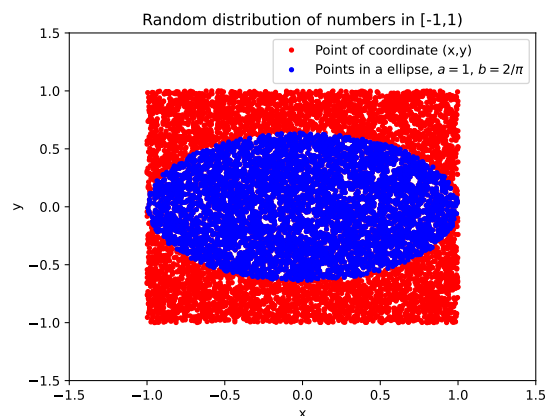


Figure 4: Plot of 10^4 random points

With the code:

```
iterations = 1000

error = np.zeros(iterations)
area = np.zeros(iterations)

for i in range(0, iterations):
    r = np.random.rand(2, N) * 2 - 1

    mask = (r[0]**2 + (np.pi * r[1] / 2)**2) < 1

    p = len(r[0, mask]) / N

    area[i] = p * A_s

area_mean = np.mean(area)
```

We arrive at:

$$A = 2.0003244 \quad \& \quad \eta = 0.5044$$

Which indeed has $\eta \geq 0.5$